# How I Met My Computer
# Technical Report

Kshatriyas

# Table of Contents

# 1.0 Introduction

The project tasked us with designing a custom multi-core processor in MATLAB Simulink and integrating it with a FreeRTOS software stack. To keep it practical and structured, we approached it as follows:

- **Extended FreeRTOS Usage:** Instead of limiting FreeRTOS to task management, we expanded it into a full system-level platform with scheduling, device drivers, synchronization, and real-time tasks.
- **Hardware Design:** Built a 5-stage RISC pipeline (fetch, decode, execute, memory, write-back) and scaled it into a multi-core system with shared memory, interconnects, UART, timers, and interrupts.
- **Software Stack:** Developed a FreeRTOS BSP including startup code, context switching, and hardware drivers. Modified the kernel for multi-core scheduling, synchronization, and interrupt handling. Validated using real-time workloads like producer–consumer and master–worker.
- **Integration and Benchmarking:** Loaded FreeRTOS binaries into Simulink instruction memory and monitored performance via UART and scopes. Measured scheduling latency, context-switch cost, CPU utilization, throughput, interrupt response, and synchronization overhead.
- **Summary:** Overall, this was a complete hardware–software co-design exercise: building a multi-core CPU in Simulink, running FreeRTOS on it, and benchmarking system performance under real-time workloads.

Classic

# 2.0 Our Approach

Our project required building a complete hardware/software co-design framework. To manage the scope, we divided the work into clear phases while ensuring close coordination between hardware and software teams. The process can be summarized as follows:

- **Planning and Scope:** We realized early that the task went beyond just modeling a processor or running an RTOS. To manage complexity, we split the work into four phases: hardware design, Simulink system modeling, FreeRTOS integration, and benchmarking. Regular communication between sub-teams ensured smooth integration.
- **Hardware Design:**
  - Selected the RISC-V ISA for its openness and modularity.
  - Implemented a 5-stage pipeline (fetch, decode, execute, memory, write-back) in Simulink.

  - Built components such as ALU, register file, and program counter using Simulink blocks and MATLAB functions.
  - Scaled from single-core to dual core and then to multi-core using a shared-bus interconnect and arbitration logic to manage memory requests and study contention effects.
  - Designed peripherals like a system tick timer (for scheduling) and a UART (for debugging/logging).

- **Software Development:**
  - Created a FreeRTOS Board Support Package (BSP) with startup routines, context-switching, and low-level drivers.
  - Extended FreeRTOS with synchronization primitives (semaphores, mutexes, barriers).
  - Adapted the scheduler for a multi-core setup.
  - Validated functionality using real-time tasks such as producer–consumer and master–worker models.

- **Integration and Co-Simulation:**
  - Loaded FreeRTOS binaries into the Simulink instruction memory and mapped the firmware into instruction memory for each processor core.
  - We refine peripheral models and adjust drivers where needed.
  - Debugging required multiple iterations due to mismatches between hardware assumptions and software needs.
  - Integration will become a joint hardware–software debugging effort.
  - The integration of the RTOS software stack over the 4-core is successful.

- **Performance Analysis and Benchmarking:**
  - We will use the system timer and custom logging to measure scheduling latency, context-switch overhead, synchronization costs, and CPU utilization.
  - We will focus on contention and synchronization overhead ratio to quantify time lost to locks/barriers.
  - We will run workloads under stress to analyze trade-offs between throughput and synchronization efficiency.

# 3.0 Implementation

## 3.0.1 Simulink

The idea is to construct a single RISC-V processor. The model can then be initiated multiple times and linked to each other via shared memory.

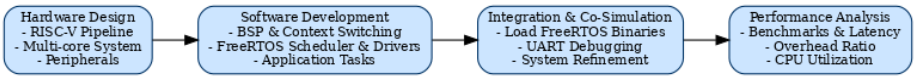


Figure 1: Workflow of our approach

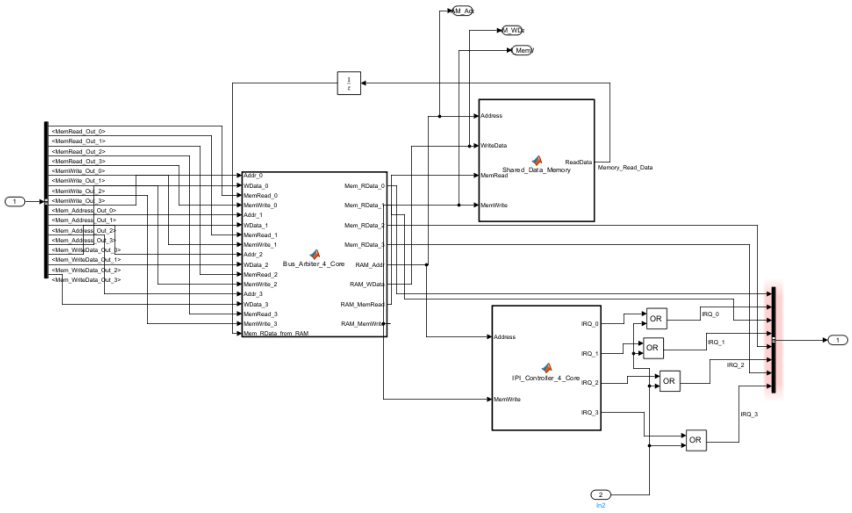


Figure 2: Diagram depicting the main hardware having bus arbiter shared memory and IPI Controller

**Implementing the RISC pipeline**

1. **Instruction Fetch :** The memory where the program instructions are stored is conveniently modelled by a Direct Lookup Table Block. The unit delay block introduces a delay of one simulation step in the signal and produces output at every integral time step. This is ideal for modelling the program counter. During each simulation step, the Mux selects the next **PC** value, choosing between **PC+4** and branch/jump target address.

2. **Instruction Decode :** The control unit is implemented as a MATLAB function block. MATLAB allows us to parse the instruction recieved from memory and produce a bus of signals. The control unit must also fetch data from registers. Registers can be implemented using the RAM block. It has two read ports to fetch data from register and one write port for the write-back stage.

3. **Execute :** ALU (Arithmetic Logic Unit) is also implemented as a MATLAB function block.

4. **Memory Access :** These steps deal with data memory and registers.

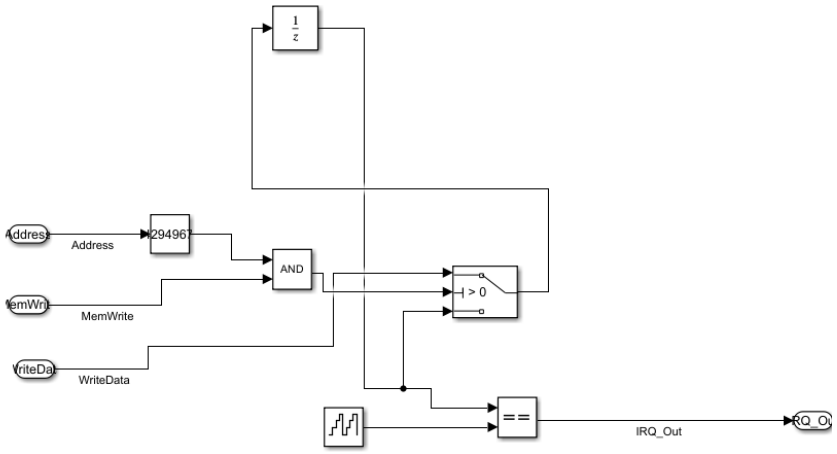


Figure 3: SysTick$_T imer$

Both are modeled as MATLAB function blocks. The data memory is implemented as a MATLAB array. Unit Delay blocks are added to model latency.

5. **Write Back:** This is the last step in the instruction cycle, Whatever data the processor has calculated or fetched needs to be stored . In this stage, the processor takes the final result and writes it into the register file.

   At the the end, the pipeline registers are implemented using unit delay blocks.Pipeline registers are placed between two stages to hold data and give instructions as the data moves from one stage to another.

### Mananging data and control hazards

The pipelining that the RISC-V architecture allows, while significantly enhancing throughput, also introduces some hazards into the system.

1. **Data Hazard:**

   Data hazards are the most common type of hazards. They happen when an instruction needs a result that a previous instruction has not executed yet. This happens because pipelining allows multiple instructions to overlap, but the result of one instruction may not be written back before the next one needs it.

   The simplest solution is stalling, where no operations are inserted until the data is ready. A NO operation helps by giving earlier instructions time to finish and make their results available, preventing later dependent instructions from using incomplete or incorrect data.

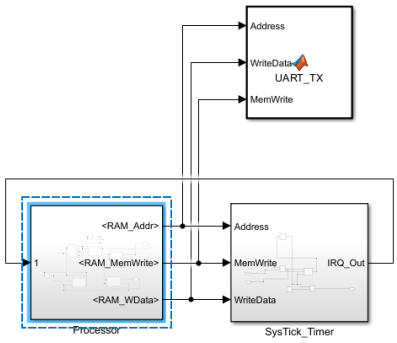


Figure 4: Shows the processors connected with the peripherals

2. **Control Hazard:**

   Control hazards occur when the processor cannot immediately determine the next instruction to fetch. They arise because the outcome of conditional decisions is only known after later pipeline stages.

3. **Jumps and branching:**

   Jumps and branching are instructions that change the normal sequential flow of a program. A jump unconditionally transfers control to a new address, while a branch conditionally changes execution based on a test. Jumps and branches create control hazards since they change the program's flow. Until the processor knows whether a branch is taken or not, it risks fetching and executing the wrong instruction.

   Our solution handles this hazard by "stalling" or "flushing". Our core always pursues a branch regardless of the branch outcome. Once the outcome is available, the instructions fetched are flushed and new instructions from the correct branch are fetched. We are flushing the pipeline before it reaches the execute state.

### Implementing multi-core

The concurrency required in a multicore CPU is readily implemented using Simulink's native features such as:

- Partitioning : Making (logical) groups of blocks within a model to carry out tasks in parallel. The groups can be made implicitly : using different sample rates in the model for each group; or they can be made explicitly by assigning all blocks in a group to a common task
- Mapping : Assigning the individual tasks to available groups.

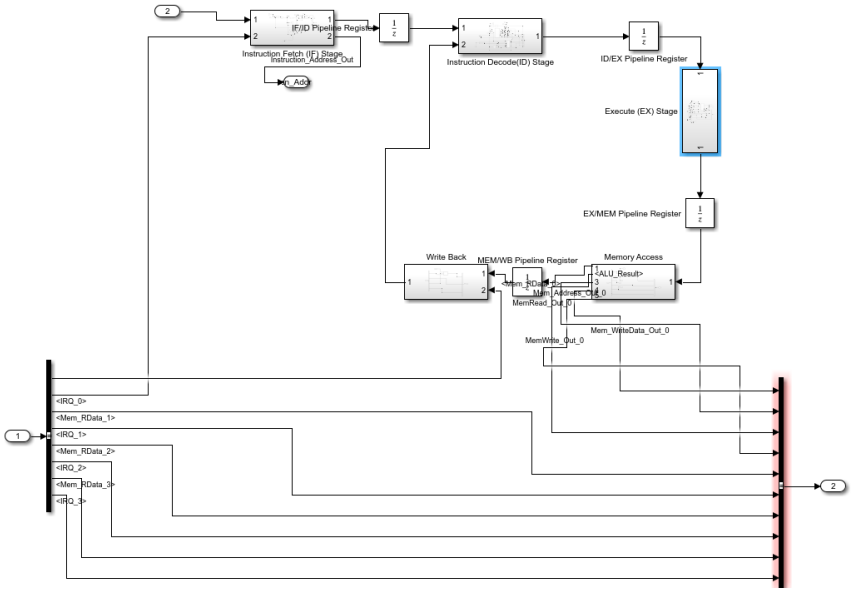


Figure 5: Describes the block diagram for data flow inside the single core.

Step 1: **Instantiating Multiple Cores :** The single core is packaged into a Subsystem Block in Simulink and copied to create a quad-core processor.

Step 2: **Shared Bus + Arbiter :** The cores need to access a shared memory through a shared bus. Arbiter decides which core gets to access or write to memory at an instant (routes requests from winning core to memory and sends response back). Arbiter is implemented as a MATLAB function block.Arbiter can be round-robin or fixed priority.

Step 3: **Shared Memory :** As stated previously, the data memory is implemented as a MATLAB array inside a MATLAB function block.Unit Delay blocks are added to model latency.

### Integrating Essential Peripherals

A system needs peripherals to interact with the outside world, it cannot work with just processors and cores.

### UART

- Universal Asynchronous Receiver-Transmitter (UART) is used to establish a communication channel between Simulink and the simulated processor so that we can monitor how the software operates within our simulated processor.

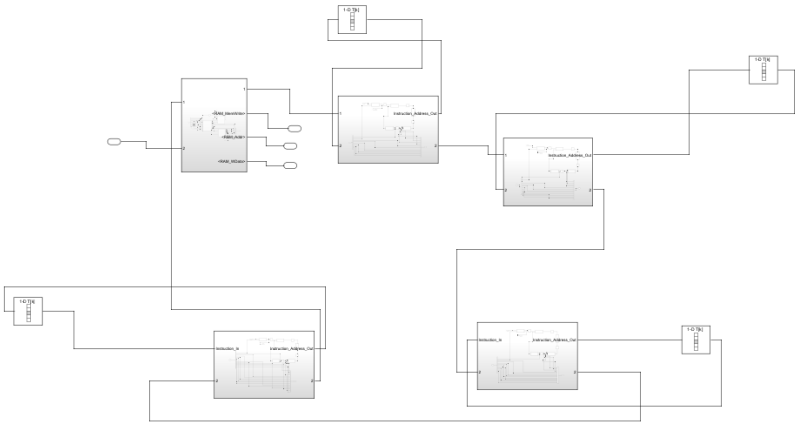

Figure 6: Inside of processor with 4 cores and the main hardware

- When the simulated processor writes a character to the Tx data register in the UART model, a MATLAB Function block can be triggered, which can use MATLAB's disp() command to print the character to the MATLAB command window or write it to a log file.
- This effectively creates a printf-style debugging console, providing a vital window into the software's execution state.

**A Programmable Timer for the RTOS System Tick**

A real-time operating system requires a periodic system tick. This is required by the scheduler kernel to know when to switch context, among many other things. This tick must be generated using hardware in a programmable fashion (software must be able to change the frequency of tick).

The timer can be modeled in Simulink with a Counter block, while a Compare to Constant block monitors whether the counter matches the value stored in a memory-mapped compare register. When a match occurs, the model generates an interrupt request (IRQ) signal and resets the counter.

Both the control and compare registers of the timer are represented as memory-mapped registers that the processor core model can access, enabling the software to configure and activate the timer.

**Interrupts – The Hardware-Software Interface**

Interrupts let peripherals signal the processor when something important happens (Timer Expiry, data received or transmitted by UART, External Input, Network Packet Arrival, Power or Battery Events ), instead of the CPU constantly polling for device status. Interrupts also make the system responsive allowing higher priority tasks to execute timely. Following are the stages though which the processor goes once an interrupt is raised :

1. Finish the current instruction.
2. Save minimal context (Program Counter and status register).
3. Temporarily disable interrupts of the same or lower priority.
4. Look up the source in the Interrupt Vector Table (IVT).
5. Jump to the matching Interrupt Service Routine (ISR).

The ISR should be short: clear the interrupt flag, do the minimum required work, and pass on heavy tasks to another task (which the RTOS scheduler can execute later or right-now depending on priority). This is called deferred handling. Once done, the processor executes a return-from-interrupt, restores the saved state, and continues normal execution.

Two constructs are required for enabling interrupts :

- **Interrupt Controllers** Handles many interrupt sources, can enable/disable them, and decides priority if multiple interrupts occur at once.
- **Interrupt Vector Table** A fixed table in memory that maps each interrupt source to its ISR, allowing fast and predictable handling.

## Hardware-Software Link

Tasks, context switches, and scheduling rely on hardware features:

- Register sets define task state.
- Timer interrupts enable preemptive RTOS scheduling.

### 3.0.2 FreeRTOS

**Overview**

This section details the software stack for the custom quad-core RISC-V System-on-Chip modeled in Simulink. The software is built upon the FreeRTOS kernel and includes a Board Support Package (BSP), custom device drivers and a real-time application layer. The entire stack is designed for co-simulation within the MATLAB Simulink hardware model.

### System Architecture and Build Process

We use a cross-compilation toolchain to build a bare-metal FreeRTOS application that runs on the quad-core processor.

- **Toolchain.** The riscv-none-elf-gcc toolchain is used for compiling C and assembly source files. The toolchain is defined in riscv-toolchain.cmake, which configures the project for a generic, bare-metalRISC-V target.
- **Build System.** CMake manages the build process. The `CMakeLists.txt` file defines all source files, include paths, compiler options and linker settings.

**Output.** The build process generates a firmware.elf file, which is then converted into `firmware.bin` and `firmware.hex` formats for loading into the Simulink model's instruction memory.

### Board support Package

BSP provides the essential low-level software that interfaces directly with the hardware.

**Startup code `(crt0.S)`** This file contains the processor's first instructions upon reset. Its responsibilities are :

- Initialize the Stack Pointer: It sets the `sp` register to the top of the stack region defined in the linker script `(__stack_top)`
- Set the Trap Handler: It loads the address of the FreeRTOS trap handler `(freeRTOS_risc_v_trap_handler)` into the `mtvec` register, ensuring all interrupts and exceptions are handled by the RTOS porting layer.
- Initialize Memory : It copies initialized global variables from ROM to RAM (`.data` section) and clears the uninitialized global variable section (`.bss` section) to zero.
- Call main : After initialization, it jumps to main C function to start the application

**Linker Script (linker.ld)** The `linker.ld` file instructs the linker how to arrange the compiled code and data into the processor's memory map.

- Memory Regions: It defines two primary memory regions: ROM (for code and read-only data) and RAM (for read-write data, the heap, and the stack).
- Section Mapping: It maps the various code and data sections (.text, .data, .bss) into the appropriate memory regions and provides symbols (_sdata, _sbss, etc.) that are used by the startup code for memory initialization.

**Interrupt Dispatcher (irq$_d ispatch.c$)** This file contains the central interrupt handler called by the FreeRTOS trap handler for external interrupts.

- Function: `freertos_risc_v_application_interrupt_handler`(mcause)
- Purpose: It reads the RISC-V mcause register to confirm a machine external interrupt has occurred. It then polls the status registers of the custom peripherals (IPI and Timer) to determine the interrupt source and calls the specific ISR for that device (`IPI_IRQHandler` or `TIMER_IRQHandler`).

### Device Drivers

The drivers provide a hardware abstraction layer (HAL) for the custom peripherals

**Timer Driver `timer.c`** This driver provides the "heartbeat" for the FreeRTOS scheduler.

- Function: `vPortSetupTimerInterrupt()`
- Purpose: Called by the kernel when the scheduler starts, this function configures the memory-mapped hardware timer to generate a periodic interrupt at the frequency defined by `configTICK_RATE_HZ`.
- Function: `TIMER_IRQHandler()`
- Purpose: This ISR is executed on every timer interrupt. It clears the interrupt flag and calls `xTaskIncrementTick()` to inform the FreeRTOS kernel that time has passed, enabling task delays and preemption.

**Inter-Core Hardware Driver `(ipi.c)`** This driver enables communication between the cores in the multi-core SoC.

- Function: `ipi_send(coreid)`
- Purpose: Allows one core to trigger an interrupt on another core by writing to a memory-mapped register.
- Function: `IPI_IRQHandler()`
- Purpose: The ISR that runs on the receiving core. It is responsible for clearing the interrupt and signaling a local task (typically by giving a semaphore) that an inter-core event has occurred.

**UART Driver`(uart.c)`:** Provides a simple, polling-based driver for serial communication, used primarily for logging and debugging.

- Function: `uart_putc(char c)` and `uart_puts(const char *s)`
- Purpose: Writes a character or a string to the UART's memory-mapped transmit register.

**FreeRTOS Kernel and Configuration**

1. Core Components The project links against the core FreeRTOS kernel source files, including:
   - tasks.c: The main scheduler implementation.
   - queue.c: The implementation for queues, semaphores, and mutexes.
   - list.c: The linked-list implementation used by the scheduler.
   - `heap_4.c`: The selected memory management scheme, providing pvPortMalloc and vPortFree.
2. Configuration (FreeRTOSConfig.h) The FreeRTOSConfig.h file tailors the kernel for this specific project. Key settings include:
   - `configCPU_CLOCK_HZ`: Sets the system clock frequency.
   - `configTICK_RATE_HZ:` Sets the scheduler tick rate to 1000 Hz (1ms).
   - `configTOTAL_HEAP_SIZE:` Allocates the size of the heap for dynamic memory allocation.
   - `configNUMBER_OF_CORES:` Configures the kernel for SMP operation on the 8-core custom processor.
   - `configUSE_MUTEXES` and `configUSE_COUNTING_SEMAPHORES:` Enabled to support the required synchronization primitives.

**Application Layer (main.c)**

The main.c file contains the high-level application logic.

- Initialization: The main function initializes the necessary RTOS primitives. For this project, it creates a queue for inter-task communication.
- Task Creation: It creates the application tasks. The implementation was changed from simple individual tasks to a producer-consumer model to satisfy the problem statement's requirement for a real-time workload.
- Scheduler Start: It calls vTaskStartScheduler() to hand over control to the FreeRTOS kernel. This function never returns.

Task implementation :

- vProducerTask: Periodically sends an incrementing value to a shared queue.
- ConsumerTask: Blocks and waits to receive a value from the queue, processing it upon receipt.

  This demonstrates a fundamental synchronization pattern.

**Simulation Setup and Integration**

To integrate the compiled FreeRTOS firmware (firmware.bin) into the Simulink hardware model, a MATLAB setup script was developed. This script automates the process of loading the binary into the instruction memory blocks of each core in the multi-core RISC-V SoC model.

- **Purpose** The purpose of this script is to:
  - Read the binary firmware produced by the RISC-V cross-compiler toolchain.
  - Map the firmware into the instruction memory of each processor core in the Simulink model.
  - Map the firmware into the instruction memory of each processor core in the Simulink model.
  - Automate the programming step for multi-core co-simulation, ensuring each core starts with the same binary image.
- **Workflow** The script performs the following steps:
  1. Read the Binary File
     - Opens the firmware.bin file generated after compilation. -
     - Reads its contents into a MATLAB vector as 32-bit unsigned integers (uint32), matching the instruction width of the processor.
  2. Model and Memory Block Mapping
     - Defines the target Simulink model name `(processor_with_peripherals)`.
     - Specifies all the instruction memory blocks `(Instruction_Memory_0 Instruction_Memory_3)` that correspond to the four processor cores.
  3. Programming Each Core
     - Iterates through all instruction memory blocks.
     - Uses the `find_system` function to locate each block inside the model.
     - Applies `set_param` to load the firmware contents into the block's Table parameter.
     - Displays a confirmation message for each core successfully programmed.
  4. Completion Message
     - Prints a summary confirming that the FreeRTOS binary has been loaded into all cores, making the model ready for simulation.
- **Key Advantages** - Automation: Eliminates manual loading of binaries into each memory block. - Scalability: Easily extendable to any number of cores by updating the instruction memory list. - Robustness: Provides warnings if a block cannot be found in the model, aiding debugging.

# 4.0 Analysis and benchmarking

### 4.0.1 CPU Scheduling Fairness

If a function (task) runs for too long, the operating system (FreeRTOS) interrupts and stops that process, switching to another task. This ensures that all tasks get equal CPU time. The mechanism is close to Shortest Time-Slice / Round Robin scheduling algorithms, where each process is given a fair share of processor cycles without starvation.

- Synchronisation Overhead : Estimated 40%
- Context Switch Overhead : Estimated 90 Clock Cycles
- Inter core synchronization efficiency : Estimated 15 clock cycles
- Interrupt Handling Response Time : Estimated 8 clock cycles

# 5.0 Conclusion

The problem demonstrated the power of hardware/software co-design, where we set out to build a multicore processor in MATLAB Simulink and integrate it with a FreeRTOS-based operating system. The challenge pushed us beyond traditional hardware modeling or software porting; instead, it required us to think holistically about how both layers interact and depend on one another to form a working system.

On the hardware side, we implemented a 5-stage RISC-V pipeline and scaled it to a multicore environment with shared memory, arbitration logic, and peripherals such as UART and timers. This provided the essential foundation for running real-time workloads. On the software side, we developed a FreeRTOS Board Support Package with startup code, context-switching support, and drivers, and then extended the kernel to handle multi-core scheduling, synchronization, and interrupts. Together, these efforts ensured that both cores and tasks could operate reliably and in coordination.

The integration phase was the most demanding yet the most rewarding. Getting FreeRTOS binaries to execute on our custom hardware, debugging through UART, and aligning hardware assumptions with software expectations required multiple iterations.

Overall, this project not only strengthened our technical foundation in processor design and real-time systems but also equipped us with the mindset to approach complex, integrated problems—a skill that will be invaluable in tackling industry-scale system design challenges.